

Guia do Documento de Ambiente Técnico

Author: Raja SP, AWS Labs

Source: <https://github.com/aws-labs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [technical-environment-guide.pt-br.pdf](#)

Propósito

Um Documento de Ambiente Técnico define as **ferramentas técnicas, padrões, restrições e preferências** que governam como um projeto é construído. É o contraponto técnico ao Documento de Visão e serve como referência vinculante durante a Fase de Construction do AI-DLC.

Este documento garante que a geração de código, o design de infraestrutura e as decisões de NFR estejam alinhados com os padrões organizacionais, políticas de segurança e capacidades da equipe. Sem ele, as etapas do AI-DLC farão perguntas extensas de esclarecimento para preencher essas lacunas, ou pior, farão suposições que exigirão retrabalho.

Quando Escrever um Documento de Ambiente Técnico

- Antes de iniciar qualquer novo projeto (greenfield)
- Antes de modificar um projeto existente onde as restrições técnicas mudaram (brownfield)
- Quando os padrões tecnológicos organizacionais foram atualizados
- Ao migrar entre provedores cloud, frameworks ou modelos de deploy

Aplicabilidade do Documento

Um Documento de Ambiente Técnico pode ter como alvo um dos dois contextos de projeto:

Contexto	Definição	Principais Diferenças
Greenfield	Sem código existente. Construindo do zero.	Todas as escolhas estão abertas. O documento define o ponto de partida.
Brownfield	Codebase existente. Adicionando, modificando ou migrando.	As escolhas são limitadas pelo que existe. O documento define o que manter, mudar ou evitar.

Estruture seu documento para o contexto aplicável. As seções abaixo são marcadas com **(Greenfield)**, **(Brownfield)** ou **(Ambos)** para indicar onde se aplicam.

Estrutura do Documento

1. Resumo Técnico do Projeto (Ambos)

```
## Resumo Técnico do Projeto

- **Nome do Projeto**: [Nome]
- **Tipo de Projeto**: [Greenfield / Brownfield]
- **Ambiente de Execução Principal**: [Cloud / On-Premises / Híbrido]
- **Provedor Cloud**: [AWS / Azure / GCP / Multi-cloud / Nenhum]
- **Modelo de Deploy Alvo**: [Serverless / Containers / VMs / Híbrido]
- **Tamanho da Equipe**: [Número de desenvolvedores]
- **Experiência da Equipe**: [Principais habilidades e níveis de experiência relevantes para as escolhas técnicas]
```

2. Linguagens de Programação (Ambos)

Defina as linguagens que o projeto deve usar, pode usar e não deve usar.

```

## Linguagens de Programação

### Linguagens Obrigatórias
[Linguagens que devem ser usadas para propósitos específicos.]

| Linguagem | Versão | Propósito | Justificativa |
|-----|-----|-----|-----|
| TypeScript | 5.x | Serviços backend, infraestrutura CDK | Expertise da equipe, segurança d
e tipos |
| Python | 3.12+ | Processamento de dados, funções Lambda | Ecossistema de bibliotecas ML |

### Linguagens Permitidas
[Linguagens que podem ser usadas se justificadas, mas não são obrigatórias.]

| Linguagem | Condições de Uso |
|-----|-----|
| Go | Aprovado para microsserviços de alto throughput onde latência é crítica |
| Rust | Aprovado apenas para componentes de nível de sistema com aprovação do tech lead |

### Linguagens Proibidas
[Linguagens que não devem ser usadas, com justificativa.]

| Linguagem | Motivo |
|-----|-----|
| PHP | Sem expertise na equipe, não alinhado com a direção da plataforma |
| Ruby | Padrão organizacional proíbe novos serviços em Ruby |

```

Adição Brownfield:

```

### Inventário de Linguagens Existentes
[Linguagens atualmente na codebase que devem ser mantidas ou migradas.]

| Linguagem | Uso Atual | Direção |
|-----|-----|-----|
| Java 11 | Serviços backend principais | Manter (atualizar para Java 21 na Fase 2) |
| JavaScript | Frontend legado | Migrar para TypeScript |

```

3. Frameworks e Bibliotecas (Ambos)

```
## Frameworks e Bibliotecas

### Frameworks Obrigatórios
[Frameworks que devem ser usados para seus respectivos domínios.]

| Framework/Biblioteca | Versão | Domínio | Justificativa |
|-----|-----|-----|-----|
| React | 18.x | UI Frontend | Padrão organizacional |
| Express | 4.x | Camada de API | Leve, familiaridade da equipe |
| AWS CDK | 2.x | Infraestrutura como Código | Alvo de deploy AWS |
| Jest | 29.x | Testes unitários | Test runner consistente entre projetos |

### Bibliotecas Preferidas
[Bibliotecas que devem ser usadas quando sua capacidade for necessária, mas não são obrigatórias se a capacidade não for necessária.]

| Biblioteca | Propósito | Usar Quando |
|-----|-----|-----|
| Zod | Validação de tipos em tempo de execução | Qualquer ingestão de dados externos ou input de API |
| Pino | Logging estruturado | Todos os serviços que emitem logs |
| Axios | Cliente HTTP | Chamadas HTTP de saída dos serviços |

### Bibliotecas Proibidas
[Bibliotecas que não devem ser usadas. Inclua a alternativa preferida.]

| Biblioteca | Motivo | Alternativa |
|-----|-----|-----|
| Moment.js | Depreciado, tamanho de bundle grande | date-fns ou Luxon |
| Lodash (completo) | Tamanho de bundle | JS nativo ou lodash-es para imports específicos |
| Request | Depreciado | Axios ou fetch nativo |

### Processo de Aprovação de Biblioteca
[Como um desenvolvedor obtém aprovação para usar uma biblioteca que não está nas listas obrigatória ou preferida?]

- [Descreva o processo de aprovação, ex.: "Envie uma solicitação de revisão técnica para a equipe de arquitetura com justificativa, verificação de licença e avaliação do status de manutenção."]
```

4. Ambiente Cloud e Serviços (Ambos)

```
## Ambiente Cloud

### Provedor Cloud
- **Provedor Principal**: [AWS / Azure / GCP]
- **Estrutura de Contas**: [Conta única / Multi-conta / Organização]
- **Regiões**: [Região(ões) primária(s) e região(ões) de recuperação de desastres]

### Lista de Serviços Permitidos
[Serviços aprovados para uso. Apenas serviços nesta lista podem ser usados sem aprovação adicional.]

| Serviço | Casos de Uso Aprovados | Restrições |
|-----|-----|-----|
| AWS Lambda | Computação orientada a eventos, handlers de API | Timeout máx. 15 min, 10GB memória |
| Amazon DynamoDB | Armazenamento chave-valor e de documentos | Capacidade sob demanda para dev, provisionada para prod |
| Amazon S3 | Armazenamento de objetos, assets estáticos | Deve habilitar versionamento e criptografia |
| Amazon SQS | Filas de mensagens assíncronas | Filas padrão preferidas; FIFO apenas quando ordem é necessária |
| Amazon CloudWatch | Monitoramento, logging, alarmes | Todos os serviços devem emitir logs estruturados |
| AWS Secrets Manager | Armazenamento de segredos | Todas as credenciais e chaves de API |
| Amazon API Gateway | Exposição de APIs REST e HTTP | HTTP APIs preferidas a REST para novos serviços |
| Amazon ECR | Registro de imagens de container | Obrigatório para todos os serviços baseados em container |
| AWS ECS Fargate | Computação em container | Preferido a ECS baseado em EC2 |
| Amazon RDS PostgreSQL | Armazenamento de dados relacionais | Aurora Serverless v2 para cargas variáveis |

### Lista de Serviços Bloqueados
[Serviços que não devem ser usados, com justificativa e alternativas aprovadas.]

| Serviço | Motivo | Alternativa |
|-----|-----|-----|
| Amazon EC2 (direto) | Preferir computação gerenciada/serverless | Lambda ou ECS Fargate |
| Amazon ElastiCache | Custo e sobrecarga operacional para a escala atual | DynamoDB DAX ou cache em nível de aplicação |
| AWS Elastic Beanstalk | Não se encaixa no workflow de IaC | CDK com ECS ou Lambda |
| Amazon Kinesis | Complexidade excede as necessidades atuais | SQS ou EventBridge |

### Processo de Aprovação de Serviço
[Como um desenvolvedor obtém aprovação para usar um serviço que não está na lista de permissões?]

- [Descreva o processo, ex.: "Envie uma Solicitação de Serviço Cloud com justificativa de negócio, estimativa de custo, revisão de segurança e plano operacional. Requer aprovação da equipe de arquitetura."]
```

5. Tecnologias e Padrões Preferidos (Ambos)

```
## Tecnologias e Padrões Preferidos
```

```
### Padrões de Arquitetura
```

```
| Padrão | Quando Usar | Quando Não Usar |
```

```
|-----|-----|-----|
```

```
| Serverless-first | Padrão para todos os novos serviços | Cargas que requerem conexões persistentes ou >15 min de processamento |
```

```
| Orientado a eventos | Workflows assíncronos, serviços desacoplados | CRUD simples sem efeitos downstream |
```

```
| Microsserviços | Domínios implantáveis de forma independente | Projetos pequenos com propriedade de equipe única |
```

```
| Monolito (modular) | Projetos de equipe única, MVPs em estágio inicial | Domínios multi-equipe ou escaláveis de forma independente |
```

```
### Padrões de Design de API
```

```
- **Estilo**: [REST / GraphQL / gRPC] - [Quando usar cada um]
```

```
- **Versionamento**: [Versionamento por prefixo de URL (v1/v2) / Baseado em header]
```

```
- **Documentação**: [Especificação OpenAPI 3.x obrigatória para todas as APIs REST]
```

```
- **Convenção de Nomenclatura**: [kebab-case para URLs, camelCase para campos JSON]
```

```
- **Paginação**: [Cursor-based preferido, offset-based aceitável para APIs administrativas]
```

```
- **Formato de Erro**: [Estrutura padrão de resposta de erro]
```

```
### Padrões de Dados
```

```
- **Armazenamento Principal**: [DynamoDB para dados de propriedade do serviço]
```

```
- **Dados Relacionais**: [RDS PostgreSQL quando consultas relacionais são necessárias]
```

```
- **Estratégia de Cache**: [Descreva a abordagem de cache]
```

```
- **Propriedade de Dados**: [Cada serviço possui seus dados; sem bancos de dados compartilhados]
```

```
### Mensageria e Eventos
```

```
- **Síncrono**: [HTTP/REST entre serviços para request-response]
```

```
- **Assíncrono**: [SQS para filas de tarefas, EventBridge para distribuição de eventos]
```

```
- **Schema de Evento**: [Descreva os padrões de schema de evento, ex.: formato CloudEvents]
```

```
### Padrões de Frontend (se aplicável)
```

```
- **Biblioteca de Componentes**: [ex.: Design system interno, Material UI, Shadcn]
```

```
- **Gerenciamento de Estado**: [ex.: React Context para local, Zustand para global]
```

```
- **Roteamento**: [ex.: React Router v6]
```

```
- **Ferramenta de Build**: [ex.: Vite]
```

6. Requisitos de Segurança (Ambos)

Requisitos de Segurança

Autenticação e Autorização

- **Método de Autenticação**: [ex.: Amazon Cognito, OIDC, SAML]
- **Modelo de Autorização**: [ex.: RBAC, ABAC, motor de política customizado]
- **Formato de Token**: [ex.: JWT com assinatura RS256]
- **Gerenciamento de Sessão**: [ex.: Expiração de token, rotação de refresh token]

Proteção de Dados

- **Criptografia em Repouso**: [Obrigatória para todos os data stores. Especifique o gerenciamento de chave KMS.]
- **Criptografia em Trânsito**: [TLS 1.2+ obrigatório para todas as comunicações]
- **Tratamento de PII**: [Identifique campos de PII, requisitos de mascaramento, políticas de retenção]
- **Classificação de Dados**: [Público / Interno / Confidencial / Restrito]

Segurança de Rede

- **Requisitos de VPC**: [Serviços que devem rodar em VPC]
- **Security Groups**: [Regras de privilégio mínimo, sem ingress 0.0.0.0/0]
- **WAF**: [Obrigatório para todos os endpoints públicos]
- **Endpoints Privados**: [Use endpoints VPC para acesso a serviços AWS onde disponível]

Gerenciamento de Segredos

- **Armazenamento de Segredos**: [AWS Secrets Manager / Parameter Store]
- **Política de Rotação**: [Rotação automática a cada N dias]
- **Política de Acesso**: [Políticas IAM de privilégio mínimo por serviço]
- **Práticas Proibidas**:
 - Sem segredos em código-fonte, variáveis de ambiente em tempo de build ou arquivos de configuração
 - Sem credenciais compartilhadas entre serviços
 - Sem chaves de acesso de longa duração

Requisitos de Conformidade

- **Padrões**: [SOC 2, HIPAA, PCI-DSS, GDPR, FedRAMP ou "Nenhum específico"]
- **Logging de Auditoria**: [Todas as chamadas de API registradas, CloudTrail habilitado, período de retenção de logs]
- **Varredura de Vulnerabilidades**: [Varredura de imagens de container, ferramentas de varredura de dependências]

Segurança de Dependências

- **Varredura de Dependências**: [Ferramenta e frequência, ex.: Dependabot semanal, Snyk no PR]
- **Política de Licenças**: [Licenças permitidas: MIT, Apache 2.0, BSD. Proibidas: GPL, AGPL]
- **Política de Atualização**: [CVEs críticas/altas corrigidas em N dias]

Framework de Conformidade de Segurança

Todo projeto deve adotar um framework de risco de segurança e documentar como o projeto aborda cada categoria de risco nesse framework. A escolha do

framework depende do domínio do projeto, do ambiente regulatório e dos padrões organizacionais.

****Selecione um ou mais frameworks e documente a conformidade por categoria:****

- ****Framework escolhido****: [Nome e versão, ex.: OWASP Top 10 (2021), NIST 800-53, CIS Controls v8, AWS Well-Architected Security Pillar, SANS Top 25, ou um framework organizacional interno]
- ****Justificativa****: [Por que este framework foi selecionado. Referencie requisitos regulatórios, contratos com clientes ou política organizacional, se aplicável.]

****Frameworks comuns por contexto:****

Contexto	Frameworks Comuns
Aplicações web e APIs	OWASP Top 10, OWASP API Security Top 10
Infraestrutura cloud-native	AWS/Azure/GCP Well-Architected Security Pillar, CIS Benchmarks
Governo / regulamentado	NIST 800-53, FedRAMP, ISO 27001
Software geral	CIS Controls v8, SANS Top 25
Interno / baixo risco	Checklist de segurança organizacional (documente aqui)

****Para cada categoria de risco no framework escolhido, documente:****

1. ****Como o projeto a aborda**** - Controles específicos, padrões e ferramentas que mitigam o risco
2. ****Justificativas de Não Aplicável**** - Se uma categoria não se aplica, explique por quê explicitamente. Não deixe categorias em branco.
3. ****Itens adiados**** - Se um controle está planejado para uma fase posterior, documente a lacuna atual e a fase alvo para remediação

****Onde colocar a matriz de conformidade detalhada:****

Para frameworks pequenos (10 ou menos categorias), inclua a matriz completa neste documento sob este título.

Para frameworks grandes (NIST 800-53, ISO 27001), crie um arquivo separado e referencie-o aqui:

- `security/[nome-do-framework]-conformidade.md`

Veja o exemplo CalcEngine para um exemplo completo usando OWASP Top 10 (2021) como o framework escolhido.

7. Requisitos de Testes (Ambos)

```
## Requisitos de Testes

### Visão Geral da Estratégia de Testes
| Tipo de Teste | Obrigatório | Meta de Cobertura | Ferramentas |
|-----|-----|-----|-----|
| Testes Unitários | Sim | Mínimo 80% de cobertura de linhas | Jest / pytest |
| Testes de Integração | Sim | Todas as interações entre serviços | Jest + Testcontainers /
pytest |
| Testes End-to-End | Condicional | Jornadas críticas do usuário | Playwright / Cypress |
| Testes de Contrato | Condicional | Todas as APIs entre serviços | Pact |
| Testes de Performance | Condicional | Quando alvos de SLA definidos | k6 / Artillery |
| Testes de Segurança | Sim | Todos os endpoints públicos | OWASP ZAP / Snyk |

### Padrões de Testes Unitários
- **Cobertura Mínima**: [80% de cobertura de linhas, 70% de cobertura de branches]
- **Política de Mocking**: [Mock dependências externas, não faça mock de lógica de negócio i
nterna]
- **Convenção de Nomenclatura**: [padrão describe/it, ex.: "describe('OrderService') > it('d
eve calcular total com imposto')"]
- **Localização dos Testes**: [Co-localizado com o código-fonte (ex.: `__tests__/\`) ou árvor
e separada (ex.: `tests/unit/\`)]

### Padrões de Testes de Integração
- **Escopo**: [Teste interações reais entre serviços, queries de banco de dados e contratos
de API]
- **Ambiente**: [Containers locais via Docker Compose / Testcontainers]
- **Gerenciamento de Dados**: [Fixtures de teste, abordagem de seed e limpeza do banco de da
dos]

### Padrões de Testes End-to-End
- **Escopo**: [Apenas jornadas críticas do usuário, não testes abrangentes de UI]
- **Ambiente**: [Ambiente de staging implantado]
- **Requisitos de Data-testid**: [Todos os elementos interativos devem ter atributos data-te
stid estáveis]

### Padrões de Testes de Performance
- **Requisitos de Baseline**: [Defina alvos de SLA: tempo de resposta, throughput, taxa de e
rro]
- **Cenários de Teste**: [Teste de carga, teste de stress, teste de duração]
- **Ferramentas**: [k6 / Artillery / JMeter]

### Gates de Testes no CI/CD
[Defina quais testes devem passar em cada etapa do pipeline.]

| Etapa do Pipeline | Testes Obrigatórios | Ação em Falha |
|-----|-----|-----|
| Pré-commit | Linting, verificação de tipos | Bloquear commit |
| Pull Request | Testes unitários, testes de integração | Bloquear merge |
| Pré-deploy (staging) | Testes E2E, testes de contrato | Bloquear deploy |
| Pós-deploy (produção) | Smoke tests, health checks | Auto-rollback |
```

8. Orientação sobre Código de Exemplo e Template (Ambos)

Esta seção diz ao AI-DLC e à equipe de desenvolvimento como fornecer, usar e manter código de exemplo ou template que estabelece as convenções do projeto.

```

## Orientação sobre Código de Exemplo e Template

### Propósito do Código de Exemplo
O código de exemplo estabelece os padrões canônicos para o projeto. Quando o AI-DLC gera código, ele deve seguir esses padrões em vez de inventar novos. Quando os desenvolvedores escrevem código, eles referenciam esses exemplos para consistência.

### Quando Fornecer Código de Exemplo
Forneça código de exemplo ou template para qualquer um dos seguintes:

- Configuração da estrutura do projeto - Layout de diretórios, nomenclatura de arquivos, organização de módulos
- Padrão de endpoint de API - Como um endpoint padrão é estruturado da rota à resposta
- Padrão de acesso ao banco de dados - Como consultas, transações e conexões são tratadas
- Padrão de tratamento de erros - Tipos de erro padrão, formato de resposta de erro, logging
- Integração de autenticação/autorização - Como a autenticação é aplicada aos endpoints
- Padrão de testes - Como um teste unitário padrão e de integração são estruturados
- Padrão de logging - Formato de log estruturado, o que registrar em cada nível
- Padrão de configuração - Como a configuração específica do ambiente é carregada
- Padrão de Infraestrutura como Código - Como um construct CDK padrão ou módulo Terraform parece

### Como Estruturar o Código de Exemplo

#### Localização
Armazene o código de exemplo em um diretório dedicado que o AI-DLC e os desenvolvedores possam referenciar:

...

raiz-do-projeto/
  examples/                                # Ou "templates/" se preferido
    api-endpoint/
      handler.ts                          # Handler de API de exemplo
      handler.test.ts                     # Teste correspondente
      README.md                           # Explica o padrão e quando usar
    database-access/
      repository.ts                       # Padrão de repositório de exemplo
      repository.test.ts
      README.md
    infrastructure/
      standard-lambda-stack.ts            # Stack CDK de exemplo
      README.md
  ...

#### Estrutura de Cada Exemplo
Cada exemplo deve incluir:

1. Código funcional - Não pseudocódigo. Deve compilar/executar.
2. Teste correspondente - Mostra como testar o padrão.
3. README.md - Explica:

```

- Qual padrão este demonstra
- Quando usar
- Quando NÃO usar
- O que personalizar vs o que manter como está
- Referências aos padrões relevantes deste Documento de Ambiente Técnico

Template de README de Exemplo

...

Exemplo de [Nome do Padrão]

O Que Este Demonstra

[Um parágrafo descrevendo o padrão.]

Quando Usar

- [Condição 1]
- [Condição 2]

Quando Não Usar

- [Condição 1 - com referência alternativa]

Inventário de Arquivos

Arquivo	Propósito
-----	-----
handler.ts	Implementação de exemplo
handler.test.ts	Padrão de teste

Guia de Personalização

Elemento	Personalizar?	Observações
-----	-----	-----
Estrutura de tratamento de erros	Não	Deve seguir o padrão do projeto
Lógica de negócio	Sim	Substitua pela lógica de domínio real
Caminho da rota	Sim	Siga as convenções de nomenclatura de API
Chamadas de logging	Não	Mantenha o formato de logging estruturado

Padrões Relacionados

- [Link para a seção de Padrões de Design de API]
- [Link para o padrão de Tratamento de Erros]

...

Como o AI-DLC Usa o Código de Exemplo

Durante a Geração de Código, o AI-DLC deve:

1. ****Ler os exemplos primeiro**** - Antes de gerar qualquer código, leia os exemplos relevantes do diretório `examples/`
2. ****Seguir padrões estabelecidos**** - Corresponda à estrutura, nomenclatura, tratamento de erros e padrões de teste mostrados nos exemplos
3. ****Não inventar alternativas**** - Se um exemplo existe para um padrão, use-o. Não crie uma abordagem diferente a menos que o exemplo explicitamente não se aplique.
4. ****Referenciar exemplos nos planos**** - Os Planos de Geração de Código devem referenciar quais

exemplos se aplicam a cada etapa

Mantendo o Código de Exemplo

- ****Atualize os exemplos quando os padrões mudarem**** - Os exemplos devem estar atualizados com este Documento de Ambiente Técnico
- ****Revise os exemplos durante o onboarding**** - Novos membros da equipe devem ler todos os exemplos antes de contribuir com código
- ****Versione os exemplos junto com o projeto**** - Os exemplos vivem no mesmo repositório e passam pelo mesmo processo de revisão que o código de produção
- ****Marque exemplos depreciados**** - Se um padrão for substituído, renomeie o diretório com um prefixo "deprecated-" e adicione uma nota apontando para o substituto

9. Seções Específicas para Brownfield

Inclua estas seções apenas para projetos brownfield.

```

## Brownfield: Inventário Técnico Existente

### Avaliação do Estado Atual
[Referencie os artefatos de Engenharia Reversa se disponíveis, ou forneça
um resumo do estado técnico atual.]

- **Linguagens Atuais**: [Liste com versões]
- **Frameworks Atuais**: [Liste com versões]
- **Infraestrutura Atual**: [Serviços cloud, modelo de deploy]
- **Cobertura de Testes Atual**: [Percentual ou avaliação qualitativa]
- **Dívida Técnica Conhecida**: [Itens principais]

### Regras de Migração e Modernização

#### O Que Manter
[Tecnologias e padrões que devem permanecer inalterados.]

| Tecnologia | Motivo para Manter |
|-----|-----|
| [Tecnologia] | [Justificativa] |

#### O Que Migrar
[Tecnologias que devem ser substituídas, com destino e prazo.]

| Atual | Alvo | Prioridade | Abordagem |
|-----|-----|-----|-----|
| JavaScript | TypeScript | Alta | Migração incremental arquivo por arquivo |
| REST API v1 | REST API v2 | Média | Novos endpoints usam v2, migrar existentes na Fase 2 |

#### O Que Remover
[Tecnologias, padrões ou dependências que devem ser eliminados.]

| Item | Motivo | Prazo de Remoção |
|-----|-----|-----|
| [Biblioteca depreciada] | [Preocupação de segurança/manutenção] | [Quando] |

### Regras de Coexistência
[Quando padrões antigos e novos devem coexistir, defina as regras.]

- **Versionamento de API durante a migração**: [Como v1 e v2 coexistem]
- **Migração de schema do banco de dados**: [Como as mudanças de schema são gerenciadas ao lado dos dados existentes]
- **Feature flags**: [Como as novas funcionalidades são controladas durante a transição]
- **Conflitos de dependências**: [Como versões de biblioteca conflitantes são gerenciadas]

```

Como Este Documento Se Alimenta no AI-DLC

Seção do Ambiente Técnico	Etapa do AI-DLC	Como é Usado
Resumo Técnico do Projeto	Detecção do Workspace	Contexto para classificação do projeto
Linguagens de Programação	Geração de Código	Seleção de linguagem e restrições de versão
Frameworks e Bibliotecas	Geração de Código, Design de NFR	Seleção de dependência e verificações de biblioteca proibida
Listas de Permissão/Bloqueio de Serviços Cloud	Design de Infraestrutura	Limites de seleção de serviço
Padrões Preferidos	Design da Aplicação, Design Funcional	Decisões de arquitetura e padrões de design
Requisitos de Segurança	Requisitos de NFR, Design de NFR	Seleção de padrão de segurança e verificações de conformidade
Requisitos de Testes	Geração de Código, Build e Testes	Estratégia de teste, ferramentas e metas de cobertura
Código de Exemplo	Geração de Código	Referência de padrão durante a geração de código
Inventário Brownfield	Engenharia Reversa, Planejamento do Workflow	Decisões de migração e regras de coexistência